Strata — Hướng dẫn Developer

Dành cho kỹ sư triển khai ticket

strata-kb v1.34.0 · 2026-11-01

1. Tài liệu này dành cho ai

Bạn viết code sản phẩm. Ticket đến tay bạn đã được bám vào cơ sở tri thức của tổ chức, kèm trích dẫn ghim vào một phiên bản cụ thể của một tài liệu cụ thể, và bạn hiện thực chúng.

Strata cho bạn **đúng section mà analyst đã trích dẫn**, ở phiên bản họ đã đọc, kèm cảnh báo nếu nó đã thay đổi kể từ đó. Nó cũng **publish ra codebase của bạn là gì**, để analyst, nhóm khác và chính bạn sau này tìm được service, dependency, endpoint và schema mà không phải đọc repository.

Repo dev có làm	Repo dev không làm
Đọc tri thức đã publish trên hub, gồm cả ticket BA đã publish	Ingest tài liệu từ bên ngoài repo
Publish tri thức về **chính code của nó**	Publish tri thức nghiệp vụ
Chạy quy trình hiện thực bằng các slash command	Tự merge hay tự ship bất cứ thứ gì (trừ khi bạn bật chế độ auto, mục 4)

Nó publish hai tài liệu:

Tài liệu	Nguồn gốc	Cần review
`<repo_id>-code`	`kb code-ingest`, tất định, không dùng model. Sinh lại khi cần từ working tree, không bao giờ commit	không
`<repo_id>-svc`	model soạn từ bằng chứng đó, bạn sửa lại	luôn luôn

`-code` là sự thật máy móc: service, công nghệ, dependency, lệnh, endpoint, schema, thư mục. `-svc` là khẳng định của con người: mỗi service thực sự *chịu trách nhiệm* về cái gì. Chúng dùng chung id section (`svc.<name>` có ở cả hai), nên BA vẽ container C4 lấy alias, nhãn và công nghệ từ `-code`, còn phần mô tả từ `-svc`.

Các khái niệm dùng chung, như các lớp L0 đến L3 và trích dẫn `kb-context`, được giải thích trong tài liệu kiến trúc. Phía analyst có trong tài liệu về BA.

Slash command. Mọi slash command trong tài liệu này đều có cho Claude Code, GitHub Copilot và Cursor, trừ các ngoại lệ sau:

- `/kb-summarize` không có prompt cho Copilot. Copilot nạp `.github/instructions/kb-summarize.instructions.md` khi bạn làm việc trong `.kb/**`. Cursor còn có thêm một rule, `.cursor/rules/kb-summarize.mdc`.

- `/kb-publish` trong Claude Code là một skill (`.claude/skills/kb-publish/`), không phải file trong `.claude/commands/`. Bạn vẫn gõ `/kb-publish`.
- Repo dev không có `/kb-ingest`, `/kb-init` hay `/kb-l3-fix`: nó không bao giờ ingest tài liệu bên ngoài.

---

2. Thiết lập lần đầu

2.1 Cài đặt và dựng khung

```
pip install "strata-kb[code]"
kb init --kind dev
```

Extra `code` (`tree-sitter` cùng `tree-sitter-language-pack`, khoảng 184 MB) là thứ cho phép `kb code-ingest` đọc route, page, job và symbol từ mã nguồn. Thiếu nó, các section đó được báo là `not scanned`, job vẫn là `partial`, SA phải hỏi bạn, và `kb doctor` cảnh báo. CI (`kb-code.yml`) tự cài extra này.

`kb init --kind dev` ghi config, cấu hình MCP, slash command và skill cho cả ba trợ lý, các workflow CI (`kb-pr-lint.yml`, `kb-code.yml`, `kb-doctor.yml`), `docs/impl/`, mẫu PR và một bộ quy ước cho mỗi ngôn ngữ phát hiện được. Dùng `--lang <tên>` (lặp lại được) cho ngôn ngữ không nhận ra được từ manifest. Chạy lại lệnh này là an toàn (xem phụ lục).

2.2 Trỏ tới hub và đăng ký repo

```
# .kb/config.yaml
kind: dev
repo_id: payments
hub: https://github.com/acme/kb-hub.git
intake: https://kb-intake.acme.com
```

- **`hub`** là nguồn đọc cho `kb query`, `kb get`, `kb resolve` và các cổng PR và lint.
- **`intake`** là cách CI publish: `kb publish` đẩy một tag `kb-publish/<ts>` và `kb-code.yml` chạy `kb code-ingest`, `kb build`, `kb ci-publish`, lệnh này xác thực với intake bằng GitHub Actions OIDC. Repo này không chứa secret nào.
- **`repo_id`** phải trùng id của repo này trong `federation/registry.yaml` của hub. Nhờ người quản trị hub thêm repo vào `repos:`; cho tới lúc đó publish bị từ chối. Branch protection trên hub cùng intake OIDC mới là ranh giới bảo mật thực sự.

Đừng bao giờ đặt token vào `hub:` hay `intake:`. File được commit, nên token đi vào lịch sử git. `kb doctor` cảnh báo; hãy viết lại URL, truyền URL có credential qua biến môi trường `STRATA_KB_HUB` (nó ghi đè file), và đổi (rotate) token. Máy nào dùng URL hub có credential cần Git 2.31 trở lên.

`hub:` và `STRATA_KB_HUB_URL` là hai giá trị khác nhau: `hub:` là URL git hoặc đường dẫn của repo hub; `STRATA_KB_HUB_URL` là địa chỉ HTTP gốc của hub, dùng cho MCP, claim và auto runner.

Nếu KB cần mang schema SQLite hoặc tag index bổ sung, hãy khai báo ở đây, không phải trong `kb-code.yml`, để mọi lần làm mới cục bộ và mọi lần chạy CI ingest như nhau:

```
code_ingest:
 db: [data/app.sqlite]   # explicit only
 tags: [payments]
```

2.3 Kết nối trợ lý của bạn

`.mcp.json` (Claude Code) và `.cursor/mcp.json` (Cursor) trỏ tới hub qua hai chỗ giữ chỗ, `${STRATA_KB_HUB_URL}` và `${STRATA_KB_HTTP_TOKEN}`. Chạy `kb mcp-setup` (trong trợ lý: `/kb-mcp-setup`). Lệnh hỏi địa chỉ HTTP gốc và token của hub, ghi cả hai vào `.env`, bảo đảm git bỏ qua file đó, rồi xác minh với hub, nên URL sai và token bị từ chối cho ra hai lỗi khác nhau. Token lấy từ người quản trị hub.

Hãy dùng prompt ẩn hoặc biến `STRATA_KB_HTTP_TOKEN` thay vì `--token`. Cờ đó để token lại trong lịch sử shell và trong danh sách tiến trình mà người dùng khác trên máy đọc được. `kb mcp-setup` cũng cảnh báo khi URL là `http://` thường tới một host không phải loopback, vì token khi đó đi qua mạng dạng rõ.

Cấu hình MCP đọc hai biến đó từ môi trường của bạn, không đọc từ `.env`, đúng lúc trợ lý khởi động. Hãy nạp file (`set -a; source .env; set +a`, hoặc dùng direnv) rồi khởi động lại trợ lý.

Một phiên bản cho mọi repo. Mọi repo chạy cùng phiên bản strata-kb với hub. Hãy nâng cấp hub trước. Sau đó, khi `kb mcp-setup` đã chạy (ở trên), chạy `kb doctor` trong repo này: nó cài phiên bản của hub, đặt các pin `strata-kb==<phiên bản của hub>` trong `.github/workflows/`, và làm mới skill cùng command. Trong CI nó không bao giờ cài, đặt lại pin hay làm mới skill; một job CI chạy `kb doctor` với `STRATA_KB_HUB_URL` và `STRATA_KB_HTTP_TOKEN` được đặt sẽ báo sai lệch phiên bản như một lỗi (không workflow dựng sẵn nào chạy nó). Khi `STRATA_KB_HUB` hoặc `--hub` được đặt, `kb doctor` bỏ qua đường MCP và việc kiểm tra phiên bản, và chỉ báo rằng phiên bản chưa được kiểm tra với hub. Các file trong `.github/instructions/` và `.cursor/rules/` (file `kb-summarize` của Copilot và Cursor, cùng rule code theo ngôn ngữ) chỉ được làm mới qua `kb init`. Nếu `pip install strata-kb` không ghim phiên bản nằm trong một job có `id-token: write`, mỗi lần chạy sẽ lấy bất cứ bản nào PyPI đang phục vụ, trong một job có quyền ghi vào hub; đó là lý do các workflow được ghim phiên bản.

2.4 Bật cổng PR và lượt quét hằng ngày

`kb init` ghi `.github/workflows/kb-pr-lint.yml` nhưng không bật được branch protection. Hãy thêm **`pr-lint` vào các check bắt buộc của nhánh**. Lint chạy trong CI tự nó không chặn merge; bắt buộc check mới chặn.

> Hãy quyết định việc này có chủ đích. Khác với cổng của repo BA, cổng này không bao giờ tự bỏ qua. Khi đã bắt buộc, nó chặn **mọi** pull request thiếu các mục yêu cầu, kể cả pull request của bot như nâng dependency hay revert.

`kb-doctor.yml` quét trích dẫn trong ticket của bạn mỗi ngày một lần và giữ một issue `kb-stale-citations`. Nó cần **variable** Actions `STRATA_KB_HUB` và, chỉ với hub riêng tư, **secret** Actions `KB_HUB_TOKEN` (một token chỉ đọc).

2.5 Hỏi xem hub có tự merge repo của bạn không

Auto-merge là công tắc theo từng repo trên trang Registry của hub, do admin hub bật. Hãy hỏi người quản trị hub xem nó đang bật chưa, và hiểu nó làm gì:

> `kb ci-publish` mở **một** pull request trên hub cho mỗi lần push, và intake dùng lại nhánh publish khi còn PR đang chờ, nên `-code` mới sinh cùng một bản sửa `-svc` đã merge sẽ đưa **cả hai** vào PR đó. Khi công tắc bật, hub merge **toàn bộ** PR ngay sau khi mở, kể cả nội dung `-svc`. Vì vậy việc review `-svc` diễn ra ở máy bạn, trong bước 4 của `/dev-code-seed`, trước khi section thành `reviewed`, không bao giờ diễn ra trên hub.

Nếu merge thất bại, PR vẫn mở và lần publish vẫn tính là xong. Khi công tắc tắt, mỗi lần push cộng thêm vào một PR hub đang chờ mà ai đó phải merge, và `kb doctor` báo khi `-code` của hub bị trễ.

2.6 Seed `-svc` cho dự án có sẵn (một lần)

Repository của bạn đã có các service chạy thật. Đừng ghi tài liệu cho chúng từ trang trắng: hãy chạy bước seed một lần.

> `/dev-code-seed`

#	Bước	Ai làm
1	**Preflight**: `hub:`, `repo_id:`, `intake:` đã đặt, repo đã đăng ký trên hub, cây sạch	agent
2	**Extract**: `kb code-ingest --scaffold-svc` sinh lại `-code` và tạo một `svc.<name>` `pending` cho mỗi service phát hiện được	agent
3	**Draft**: `kb summarize <repo_id>-svc` viết một đoạn trách nhiệm cho từng service	agent
4	**Review**: đối chiếu từng bản nháp với bằng chứng code và sửa lại	**bạn**
5	**Approve**: `kb approve <repo_id>-svc` chuyển các section đã sửa sang `reviewed`	bạn
6	**Flows** (tùy chọn): thêm các section `flow.<name>` cho luồng nghiệp vụ đi qua nhiều service	bạn
7	**Validate và publish**: `kb build` không kèm `--allow-pending`, rồi `kb publish --pr`	bạn

> **Bản nháp của model là bản nháp, không phải sự thật.** Nó có thể nêu sai luồng, sót một trách nhiệm hoặc nói quá một trách nhiệm. Đừng bao giờ approve một section bạn chưa đọc và chưa sửa: approve mà không đọc là phá hỏng cổng kiểm soát. Khi một trách nhiệm chạm tới một chuẩn nghiệp vụ, hãy trích `doc-id §section` từ hub thay vì diễn đạt lại quy tắc.

Dành 10 đến 15 phút cho mỗi service ở bước 4; đây là chi phí một lần. Bước 6 cứ bỏ qua thoải mái: thiếu một flow vẫn tốt hơn đoán một flow. Dùng `kb publish --pr` cho lần publish đầu tiên này: chưa có merge nào xảy ra, nên không có tag nào để `kb-code.yml` phản ứng, và `kb publish` thường sẽ chờ tới mười phút cho một lượt CI không bao giờ chạy.

Để sửa một service, hãy sửa tay section của nó. `kb summarize <repo_id>-svc --redo --include-reviewed` xóa nội dung và chữ ký duyệt của mọi section đã reviewed trong tài liệu `-svc` (thêm `--section <id>` để chỉ reset một section) rồi tóm tắt lại ngay trong cùng lệnh; đoạn văn bạn đã sửa chỉ lấy lại được từ git.

---

3. Công việc hằng ngày

3.1 Hiện thực một ticket

Mọi điểm vào đều là slash command. Chọn điểm vào khớp với tiến độ hiện tại; mọi phase đều tiếp tục được từ trạng thái nguội, vì trạng thái nằm ở `docs/impl/<id>/`, nhánh và PR, không bao giờ nằm trong cuộc trò chuyện.

Tình huống	Chạy	Kết quả
Ticket mới, chưa làm gì (ticket nhỏ tự thu gọn)	`/dev-implement-ticket <ticket>`	claim ticket, lấy về, kiểm tra, rồi đi qua design, plan, execute, handover
Design đã duyệt, chưa có plan	`/dev-plan <id>`	`docs/impl/<id>/plan.md`
Plan đã duyệt, hoặc đang execute dở	`/dev-execute <id>`	code, từng task một
Code viết tay, cần bản mô tả PR	`/dev-handover <id>`	nội dung PR, các dòng lịch sử `kb svc note`
Không biết ticket đang ở đâu	`/dev-implement-ticket <id>`	phát hiện đã đi tới đâu và đề xuất bước tiếp
Câu hỏi kỹ thuật cần bằng chứng	`/dev-sa <câu hỏi>`	một quyết định kèm bằng chứng `path:line` / `doc §section`
Chỉ muốn kiểm tra một trích dẫn	`kb resolve <file>`	`ok`, `stale` hoặc `broken` cho từng tham chiếu

Chỉ cần id ticket là đủ. Bước Intake lấy ticket **từ hub**: BA publish mọi ticket đã sẵn sàng trong tài liệu `<repo_id>-plan`, nên hub là nguồn sự thật.

```
kb ticket fetch M-checkout-US3          # or <repo>:<id> if several repos publish it
kb claim take M-checkout-US3            # one Dev per ticket; exit 3 = taken
```

`kb ticket fetch` ghi `docs/impl/<id>/ticket.md` (`--out` để đổi chỗ) và ghi đè nó ở mỗi lần lấy. Bản sao này suy ra được từ hub, nên `docs/impl/.gitignore` bỏ qua nó: đừng bao giờ commit và đừng bao giờ sửa nó. Nếu ticket sai, BA sửa trong repo BA. Lệnh thoát với mã `1` khi không có plan đã publish nào chứa id đó (nhờ BA publish) hoặc có nhiều plan chứa (viết rõ `<repo>:<id>`). Bản trên hub không có `## Definition of Ready`: có mặt trên hub nghĩa là BA đã tick mọi ô.

Ngay sau Intake, luồng chạy `kb ticket check docs/impl/<id>/ticket.md` (bỏ qua khi ticket không có `## Technical grounding`):

Kết quả	Ý nghĩa	Làm gì
PASS, `[note] ... still valid`	Không có gì ticket trích dẫn bị đổi	Làm tiếp
PASS, `[warn] ... review the diff`	Một section được trích dẫn đã đổi, mọi tên được trích dẫn vẫn còn	Đọc diff; làm tiếp trừ khi nó đổi nghĩa ticket
FAIL, `re-run /sa-ticket-ground`	Một id, cột, route, lệnh hay file được trích dẫn không còn	Dừng. Đó là điểm chặn của BA, SA của họ sẽ bám lại ticket
FAIL, `checked against the local -code`	Hub không với tới được hoặc chưa có `-code`; có thể nhánh của bạn là nguyên nhân	Chạy lại khi hub với tới được
FAIL, không có dòng re-ground	Một quyết định còn mở hoặc một `[NEW: D<k>]` chưa quyết	Báo BA rồi dừng

Menu Ticket của hub (`/ui/tickets`) và `kb mission next --from-hub [--repo <ba-repo>]` hiển thị từng story là **done**, **ready**, **not ready** hoặc **blocked**. Hãy chọn story **ready**. Ticket `draft` mà lý do nói mission cha đã được sửa thì chưa publish: chờ BA, và đừng hiện thực từ bản cũ.

3.2 Bốn cổng do người duyệt, năm lượt review do agent

```
/dev-implement-ticket <ticket>     intake, questions, check, resolve, ground
       |
       +-> dev-design    -- A1 review -- GATE 1: you approve the design
       +-> dev-plan      -- A2 review -- GATE 2: you approve the plan
       +-> dev-execute      A3 per task, A4 at the end (repeatable, resumable)
       +-> dev-handover  -- A5 merge-risk review -- GATE 3: you open the PR
                                                    GATE 4: you merge it
```

Không có gì được merge hay ship nếu thiếu con người. Agent không bao giờ tự mở PR hay merge. Trước mỗi cổng trong ba cổng đầu, một agent khác đã đọc lại công việc trong ngữ cảnh riêng của nó, không bao giờ là chính agent đã làm ra nó:

	Ở đâu	Xem xét gì
A1	`dev-design`, trước GATE 1	design có khớp ticket không
A2	`dev-plan`, trước GATE 2	plan có test-first và chạy được không
A3	`dev-execute`, mỗi task	diff có đáp ứng spec không, code có tốt không
A4	`dev-execute`, sau task cuối	nhánh hoàn chỉnh có khép kín ticket không
A5	`dev-handover`, trước GATE 3	merge vào nhánh mặc định có an toàn không

Tiêu chí nằm trong `docs/pr-review-rubric.md`, được `docs/pr-review-rubric.local.md` ghi đè. Giới hạn số vòng nằm trong các skill; một finding còn mở sau vòng cuối sẽ dừng luồng và chuyển cho bạn. Nếu trợ lý của bạn không điều phối được subagent (mặc định với wrapper Copilot và Cursor), lượt review chạy như một lượt đơn và chỉ là **báo cáo, không phải cổng**: phase dừng và giao kết quả cho bạn. `dev-execute` sắp thứ tự task bằng `kb plan waves docs/impl/<id>/plan.md` và chạy các task độc lập song song trong các lane git worktree. `kb plan waves` báo lỗi khi có file dùng chung mà không có phụ thuộc, khi có chu trình, khi một task không liệt kê đường dẫn nào dưới **Files**, khi thiếu dòng `Depends on:` hoặc dòng đó nêu một task không tồn tại, hoặc dòng `Serial resource:` rỗng; task có dòng `Serial resource:` được in sau `serial:` và chạy một mình, không bao giờ trong lane song song.

3.3 Câu hỏi cho BA

`dev-design` và `dev-plan` không bao giờ giải quyết một nghi ngờ bằng cách đoán. Mỗi nghi ngờ được gắn nhãn `business` hoặc `technical`:

- **Kỹ thuật** (xây thế nào, đặt ở đâu) chuyển cho `dev-sa`, kiến trúc sư giải pháp của repo này. Nó đọc mã nguồn và các tài liệu `-code`, `-svc` và miền của hub, và quyết định được ghi vào bản nháp dưới dạng `- SA-<n>: <decision> — why — evidence`. Bạn có thể tự gọi: `/dev-sa <câu hỏi>`.

- **Nghiệp vụ** đi tới BA qua Q&A của hub (MCP `kb_question_ask`) và xuất hiện dưới dạng `- Q<id> — <question> — blocks: <AC ids|none>`. BA trả lời bằng `/ba-answer-questions`.

Một câu hỏi chặn một tiêu chí chấp nhận (AC) sẽ dừng cổng: skill đưa bạn đường dẫn Q&A của hub để dán vào chat. Ở mỗi lần vào, `/dev-implement-ticket` đọc các câu hỏi của ticket và hiển thị `BA-Q: none | open Q<ids> | waiting Q<ids>`; câu hỏi *waiting* chỉ để biết, câu hỏi *open* đang chặn thì dừng luồng. Q&A chỉ hoạt động khi MCP trỏ tới máy chủ HTTP của hub (`kb mcp-setup`).

3.4 Sửa một bug

Bug do BA publish có dạng `BUG-<n>`: không có story, không có tiêu chí chấp nhận, xong khi các bước của nó không còn tái hiện kết quả sai. `/dev-fix-bug BUG-<n>` chạy:

1. **Intake**: `kb ticket fetch BUG-<n>` (cũng sao chép ảnh chụp màn hình vào `docs/impl/<id>/assets/`); nhánh `fix/<BUG-id>`, hoặc `hotfix/<BUG-id>`.
2. **Reproduce**: một test fail cho thấy kết quả Actual. Không có test này thì không fix.
3. **Root cause**: các giả thuyết trong `docs/impl/<id>/rootcause.md`; ba giả thuyết thất bại thì luồng dừng lại để bàn về kiến trúc với bạn.
4. **Fix**: một thay đổi tại nguyên nhân gốc, toàn bộ test xanh.
5. **Handover**: `/dev-handover` với quy tắc cho bug; test tái hiện và nguyên nhân gốc thay thế design và plan.

3.5 Nhiều Dev trên một repo: claim

Một **claim ticket** trên hub giữ hai Dev khỏi làm cùng một ticket. `/dev-implement-ticket` và `/dev-fix-bug` chạy `kb claim take <id>`, và mọi phase sau đó làm mới nó.

- Claim ghi tên người giữ (`<git user.name>@<hostname>`, hoặc `STRATA_KB_CLAIM_AS` hay `--as`) và một phase (`claimed`, `design`, `plan`, `execute`, `handover`, `waiting-qa`, `review`, `publishing`). Dev thứ hai nhận `taken by <holder> (<phase>, updated <x> ago)` và mã thoát `3`: chọn ticket khác hoặc nhờ admin hub giải phóng.
- Theo mặc định nó hết hạn sau 4 giờ không cập nhật. Khi bạn đã mở PR, chạy lại `/dev-handover <id>`: nó đặt `review` kèm URL của PR. Claim ở `review` hoặc `publishing` có PR, hay claim mà auto runner đánh dấu `stopped`, không bao giờ hết hạn. Nếu claim của chính bạn bị giải phóng khi bạn đang làm, lần cập nhật kế tiếp thoát với mã `3` (`lost: <reason> by <who>`) và skill dừng mà không push.
- `kb claim list [--json]`, `kb claim release <ticket> [--reason TEXT]` và `kb claim update <ticket> --phase <p>` quản lý claim. Các lệnh này cần `STRATA_KB_HUB_URL` và `STRATA_KB_HTTP_TOKEN`.

3.6 Publish sau khi merge

Push lên nhánh mặc định không publish gì; intake chỉ nhận publish từ tag `kb-publish/*`. Sau khi PR của bạn merge, hãy publish từ nhánh mặc định của bạn (`<default>`):

```
git fetch origin && git switch <default> && git pull --ff-only origin <default>
kb publish
```

Trong repo dev, `kb publish` từ chối chạy từ checkout đang tụt sau `origin/<default>`, vì snapshot sẽ làm rơi dòng `hist` mới hơn của máy khác. Các thông báo là `behind origin/<default> — pull, then kb publish`, `no origin remote — ...` và `could not fetch origin/<default>: ...`. Squash merge để nhánh ticket tụt lại phía sau, đó là lý do bạn chuyển nhánh trước. Hub phản ánh commit khi PR của nó merge. `kb publish` và `kb ci-publish` cảnh báo có bao nhiêu section được ship khi chưa review (`--require-reviewed` biến cảnh báo đó thành lỗi).

3.7 Làm việc với trích dẫn

```
kb resolve tickets/ABC-123.md    # ok | stale | broken, per ref
kb diff hr-handbook --against <rev>
kb get hr-handbook 4.12 --level l3
```

Trạng thái	Ý nghĩa	Làm gì
`ok`	Nội dung đã ghim vẫn khớp với bản đang publish	Hiện thực theo nó
`stale`	Nguồn đã được sửa sau khi ticket được viết	Chạy `kb diff`, rồi hỏi analyst xem các tiêu chí chấp nhận còn đúng không. **Đừng lặng lẽ hiện thực bản mới.**
`broken`	Tham chiếu không còn phân giải được	Báo lên. Ticket không hiện thực được như đã viết

Mọi điểm vào đều kiểm tra lại độ mới trước, vì hub có thể đã publish kể từ phiên làm việc trước của bạn. Khi một giá trị trong code của bạn đến từ một chuẩn (định dạng, enum, ngưỡng, độ dài), hãy lấy nó từ section đã phân giải ở phiên bản đã ghim và để lại một comment trích dẫn ngay cạnh.

3.8 Giữ tri thức luôn mới

Không tài liệu nào đòi hỏi kỷ luật mới theo từng ticket từ bạn.

- **`-code` không cần làm gì.** Nó không bao giờ được commit (`.gitignore` có `.kb/*-code/`). `kb svc note`, `kb build`, `kb doctor` và `kb publish` sinh lại nó trước khi chạy khi nó thiếu, cũ hơn `HEAD`, hoặc các file đang được theo dõi có thay đổi (`--no-refresh` bỏ qua việc đó). CI không bao giờ truyền `--scaffold-svc`, vì CI không được tạo nội dung `pending`.
- **`-svc` tích lũy ở handover.** `dev-handover` chạy `kb svc note payments-api --ticket ABC-123 --title "Add refund endpoint"` cho mỗi service mà ticket đã chạm tới. Lệnh thêm một dòng `hist.<service>`, idempotent; đừng bao giờ sửa `hist.*`.
- **Trôi lệch thì được báo, không tự sửa.** Khi bằng chứng làm mới của một service đã `reviewed` thay đổi, `kb code-ingest --scaffold-svc` in `stale-risk: svc.<name>` và để nguyên phần tóm tắt.

---

4. Các chế độ tự động hóa

Chế độ full-auto (tùy chọn). Một công tắc cho mỗi repo `ba` và `dev`, cho phép hai repo tự chạy, trên máy của bạn, cho tới khi mọi ticket trên hub xong. Không bước nào chờ con người: agent tự review công việc của mình, duyệt GATE 1 và GATE 2, chọn phương án tốt nhất, ghi lý do vào `docs/impl/<id>/auto-decisions.md` (được chép vào `## Notes` của PR), mở PR, sửa CI cho tới khi xanh, merge rồi chạy `kb publish`. Nó được xây cho **một repo BA và một repo Dev trên mỗi hub**. Khi công tắc tắt thì không có gì thay đổi.

4.1 Bảo mật: đọc phần này trước

Danh sách từ chối quyền mà `kb init` ghi ra (phụ lục) chỉ là phòng thủ nhiều lớp, không phải ranh giới. Một lượt chạy không người giám sát tự merge pull request của chính nó và publish lên hub mà không có ai review. Một ticket mà agent hiểu sai sẽ đi thẳng vào `main` và vào hub. `kb auto on` tự in cảnh báo này: "Auto mode merges pull requests as admin, with no review. Turn it on only for a repo where that is acceptable."

Nó merge bằng `kb pr merge` (GitHub REST), dùng quyền của tài khoản `gh` của bạn; `--admin` được chấp nhận cho tương thích với `gh` và không đổi gì. Nếu tài khoản đó là admin của repo và branch protection không bao gồm administrators, các lần merge đó vượt qua protection. Vì vậy, trước khi chạy, chủ repo **phải** bật GitHub branch protection trên `main`:

- chặn force push và xóa nhánh;
- **bao gồm administrators**;
- giữ check CI là **required status check**: job(s) `test-<lang>` của `kb-test.yml` (hoặc của workflow mang marker `kb-test`). Branch protection chọn check theo tên job. Nếu không có check bắt buộc, "green CI" chỉ là quy ước mà agent tự tuân theo;
- **đừng** bắt buộc review chấp thuận: agent không thể tự approve PR của mình, và merge khi đó bị từ chối. Nếu bạn vẫn bắt buộc review, hãy loại administrators khỏi quy tắc đó và chấp nhận rằng agent vượt qua nó trên `main`.

Danh sách từ chối cũng không nhìn thấy nhánh hiện tại: working tree của agent không bao giờ được đứng trên `main` khi đã có commit. Các skill tạo nhánh trước; runner không canh việc này. Runner không bao giờ dùng `--dangerously-skip-permissions` hay `bypassPermissions`.

4.2 Thiết lập một lần

1. Một mission và các ticket của nó đã có trên hub (phía BA, làm tay hoặc ở chế độ auto).
2. `intake:` đã đặt trong `.kb/config.yaml`, và trang Registry của hub có **auto-merge bật** cho repo này, để dòng `hist` được ghi và các story đạt `done`. Điều này áp dụng cho cả repo BA.
3. Chạy lại `kb init --kind dev`. Nó gộp allowlist quyền, gồm các lệnh test, install, lint và build của `langs:` ghi trong `.kb/config.yaml`, vào `.claude/settings.json`, và làm mới skill cùng `/dev-test-workflow`.
4. Export `STRATA_KB_HUB_URL` và `STRATA_KB_HTTP_TOKEN`, và có CLI `claude` trong `PATH`, **đăng nhập bằng gói đăng ký Claude**. Runner không bao giờ dùng API key.
5. Giữ `main` làm nhánh mặc định; các skill chạy `git fetch origin main`.

4.3 Chạy và dừng

Việc cần làm	Lệnh	Kết quả
Bật công tắc	`kb auto on`	in cảnh báo, bật `auto.enabled`
Commit nó	commit và push `.kb/config.yaml` và `.claude/settings.json` lên nhánh mặc định	`kb auto run` từ chối cho tới khi bạn làm vậy
Khởi động vòng lặp	`kb auto run` (`--once`, `--model`, `--effort`), hoặc `/kb-auto run`	`/kb-auto run` làm cả ba bước và chạy vòng lặp tách rời; log ở `.kb/auto/run.log`
Xem đang ở đâu	`kb auto status [--json]`, `/kb-auto status`	công tắc, task, số lần thử, câu hỏi đang chờ, lý do dừng
Dừng	`kb auto off`, `/kb-auto off`	dừng sau task hiện tại
Chờ kết quả	`kb auto wait [--max 6600] [--interval 30]`	một dòng, dùng cho bộ theo dõi trên cloud

`kb auto run` từ chối, và in mọi lý do, khi công tắc tắt, `.kb/config.yaml` hoặc `.claude/settings.json` chưa được theo dõi hay đang bẩn, `auto.enabled: true` thiếu trong bản config của nhánh mặc định, `intake:` rỗng, hoặc registry của hub không đọc được, không liệt kê repo này, hoặc thiếu `auto_merge: true` cho nó. Mã thoát: `0` xong, `2` đã dừng, `1` bị từ chối hoặc lỗi, `130` bị ngắt.

4.4 Nó làm gì và khi nào dừng

Mỗi task một tiến trình `claude -p` mới. Theo thứ tự:

1. Nếu chưa có workflow nào chạy test của repo, `/dev-test-workflow` viết và merge `kb-test.yml`, để "green CI" nghĩa là test đã chạy.
2. Bug trước (`/dev-fix-bug`).
3. Ticket `ready` đầu tiên không thuộc mission nào và không phải bug (`/dev-implement-ticket`).
4. Story `ready` đầu tiên theo thứ tự mission mà các phụ thuộc đã `done`.

Khi một câu hỏi nghiệp vụ chặn ticket, Dev hỏi BA qua Q&A của hub và chờ; khi đã trả lời, cùng ticket đó chạy tiếp trong một tiến trình mới. Nó dừng sau hai lần thử hỏng ở một ticket (CI đỏ sau hai vòng sửa thì dừng ngay); hãy sửa nguyên nhân rồi chạy lại `kb auto run`. Một lệnh nằm ngoài allowlist kết thúc task bằng `failed command-denied <command>`: hãy thêm nó vào `permissions.allow` trong `.claude/settings.json` bằng tay. Chờ vì chạm giới hạn sử dụng (15, 30, 60 phút) không phải thất bại. Nó kết thúc, mã thoát `0`, khi mọi story, ticket không có mission và bug đều `done`. Một tiến trình task mang `STRATA_KB_AUTO_TASK=1`, dưới biến này `kb auto on`, `off` và `run` từ chối (`status` vẫn chạy), nên một task không thể vừa bật công tắc vừa khởi động vòng lặp thứ hai.

Prompt của runner yêu cầu mỗi task kết thúc câu trả lời bằng một dòng: `AUTO-RESULT: done`, `AUTO-RESULT: blocked-qa Q<id>[,Q<id>...]` hoặc `AUTO-RESULT: failed <reason>`, và không có gì sau nó. Dòng thiếu hoặc sai dạng được tính là một lần thử thất bại (`no result line`, `bad result line`). Skill bạn tự viết cho runner phải kết thúc theo cùng cách.

4.5 Claude Code cloud

`/kb-auto run` cũng chạy trong một phiên Claude Code cloud (nhận biết qua `CLAUDE_CODE_REMOTE`; `STRATA_KB_CLOUD=1|0` ép buộc). Đặt `STRATA_KB_HUB_URL` và `STRATA_KB_HTTP_TOKEN` trong môi trường cloud. Hub được đọc trực tiếp qua API của nó và không clone gì; các lệnh `kb` khác đọc hub từ chối và chỉ sang các công cụ MCP `strata-kb`. `kb pr` nói chuyện với GitHub REST API thay vì `gh pr`, vì GraphQL bị chặn ở đó. Vì không ai bấm được "trust this folder", `kb auto run` ở đó đánh dấu thư mục gốc repo là tin cậy trong `~/.claude.json`; các khóa khác được giữ nguyên và file không phân tích được thì bị bỏ qua.

Nhánh của phiên không push được lên nhánh mặc định. Khi công tắc của nhánh mặc định tắt, `/kb-auto run` mở một PR `auto/switch` chỉ đổi `auto.enabled`, chờ check xanh, rồi **tự merge PR đó, báo cho bạn trước**, bằng `kb pr merge auto/switch --squash --admin --delete-branch`. Lời khuyên branch protection ở 4.1 áp dụng y nguyên: lần merge đó cũng không có review. `/kb-auto run` chạy lại được: nó tự gỡ worktree và nhánh `auto/switch` cục bộ còn sót. PR chuyển công tắc bị bỏ mở sau một lượt chạy thất bại (check đỏ, lỗi merge) và nhánh `auto/switch` từ xa của nó phải được đóng và xóa trên GitHub bằng tay trước; skill dừng lại và nói rõ, và không bao giờ force-push. **Phiên phải luôn mở** suốt lượt chạy. Chưa được kiểm chứng trên cloud thật: phiên kéo dài hơn ba giờ, push các nhánh `auto/*`, merge vượt qua một review bắt buộc, và `gh run` qua REST.

---

5. Xử lý sự cố



6. Phụ lục

6.1 Khóa config và biến môi trường

Khóa hoặc biến	Ý nghĩa
`kind`, `hub`, `repo_id`, `intake`	xem 2.2. Đừng bao giờ đặt token vào `hub:` hay `intake:`
`langs`	ngôn ngữ cho bộ quy ước và allowlist của chế độ auto; đặt bởi `--lang` hoặc tự phát hiện
`code_ingest.db`, `code_ingest.tags`	file SQLite (chỉ khi nêu rõ) và tag index bổ sung cho mọi lần làm mới `-code`
`auto.enabled`, `auto.qa_interval`, `auto.idle_interval`, `auto.hotfix_base`, `auto.claim_heartbeat`	chế độ auto; `hotfix_base` rỗng = nhánh mặc định (hiện các skill giả định `main`); `claim_heartbeat` là số giây giữa các lần làm mới claim, mặc định `900`
`kb init` chạy lại	tạo file còn thiếu và làm mới các file dựng khung; `.kb/config.yaml`, `.kb/index.yaml` và `.claude/settings.json` được bảo vệ (`--force` ghi đè chúng; nếu không, `.claude/settings.json` được gộp); các file `*.local.md` không bao giờ bị đụng tới
`.kb/index.yaml` `llm:`	`runner` (`auto`, `claude`, `copilot`, `none`), `model`, `effort`, `max_workers`, `timeout`, dùng bởi `kb summarize <repo_id>-svc`
`STRATA_KB_HUB`	ghi đè `hub:`; mang URL có credential trong CI (một variable Actions)
`STRATA_KB_HUB_URL`, `STRATA_KB_HTTP_TOKEN`	địa chỉ HTTP gốc và bearer token của hub: MCP, claim, auto runner
`KB_HUB_TOKEN`	secret Actions, token chỉ đọc, chỉ cho hub riêng tư (`kb-doctor.yml`)
`STRATA_KB_CLAIM_AS`	tên người giữ claim
`STRATA_KB_AUTO_TASK`	do runner đặt trên các tiến trình task; `kb auto on`, `off` và `run` từ chối khi có nó
`STRATA_KB_CLOUD`, `CLAUDE_CODE_REMOTE`	ép buộc hoặc nhận biết Claude Code cloud

Mã thoát: `0` ok, `1` lỗi, `2` trích dẫn stale (hoặc lỗi cách dùng), `3` claim bị người khác giữ hoặc bị mất, `8` `kb pr checks` còn đang chờ.

6.2 Thư mục làm việc và những gì bị git bỏ qua

Đường dẫn	Chứa	Commit
`docs/impl/<id>/design.md`, `plan.md`	design và plan, mỗi file có bảng `## Review record`	có
`docs/impl/<id>/ticket.md`	ticket lấy từ hub	không, đừng bao giờ sửa
`docs/impl/<id>/context.md`, `review/`, `assets/`	cache ngữ cảnh, tệp review, ảnh chụp màn hình bug (có thể lộ dữ liệu cá nhân)	không
`.kb/*-code/`, `.kb/auto/`, `.env`	`-code` suy ra, trạng thái của auto runner, thông tin đăng nhập MCP	không

`kb init` và `kb impl tidy` chuyển bố cục phẳng cũ (`docs/impl/<id>-plan.md`, ...) vào `docs/impl/<id>/` mà không ghi đè; hãy commit các lần đổi tên. Đừng bao giờ commit `.kb-qa/`: đó là kho Q&A của hub, và nó bị git bỏ qua trên hub.

6.3 `kb code-ingest` trích xuất gì

```
kb code-ingest                          # defaults are usually right
kb code-ingest --db data/app.sqlite     # name a database explicitly
kb code-ingest --json                   # machine-readable report
```

Mười một extractor, tổ chức theo loại artefact chứ không theo ngôn ngữ. Bảy cái đầu đọc file build, deploy và hợp đồng; bốn cái cuối đọc mã nguồn bằng tree-sitter.

Extractor	Tiền tố section	Đọc
`services`	`svc.<name>`	file compose, Dockerfile, manifest k8s, file solution và workspace
`deps`	`dep.<ecosystem>`	manifest của Python, JavaScript, Java, .NET, Go, PHP, Rust, Swift, Dart
`commands`	`cmd.<purpose>`	script package, Makefile, tox, shell script, bước run trong CI
`tree`	`struct.tree`	các file Git theo dõi, nên virtualenv bị bỏ qua không bao giờ xuất hiện
`schema`	`db.<table>`	migration SQL, Prisma, Alembic, EF, và file SQLite được nêu rõ
`integrations`	`int.<name>`	chỉ **khóa** biến môi trường
`api`	`api.<tag>`	hợp đồng OpenAPI và Swagger nằm trong repo
`routes`	`route.<slug>`	route HTTP khai báo trong mã nguồn (cần `[code]`)
`pages`	`page.<slug>`	route trang UI (router theo file hệ thống chạy được không cần `[code]`)
`jobs`	`job.<slug>`	job theo lịch của các scheduler framework (cần `[code]`)
`symbols`	`sym.<slug>`	tên được export, thành viên enum và object, CSS custom property (cần `[code]`)

Với mỗi service compose, section `svc.<name>` của `-code` còn liệt kê các volume có tên (bind mount không phải volume có tên nên bị bỏ), lệnh healthcheck trên một dòng (`disabled` khi service tắt nó, `none` khi không có) và các device được đặt trước (`<driver>:<caps>`, lấy từ `deploy.resources.reservations.devices`).

`tree` còn ghi `struct.files.<dir>` (mọi file được theo dõi dưới một thư mục cấp cao nhất), và mọi extractor báo cáo vào `struct.coverage`. `kb code-ingest` thoát với mã `1` khi không extractor nào trong `services`, `deps`, `commands`, `schema`, `integrations` và `api` tìm thấy gì: chỉ một danh sách thư mục là cấu hình sai, không phải tri thức.

Framework route được hỗ trợ: NestJS, Express, Next.js, Nuxt, Spring Boot, Quarkus, Micronaut, ASP.NET Core, FastAPI, Flask, Django, Gin, Laravel, Symfony, Actix Web, Axum, Rocket, Vapor.

Pin `tree-sitter-language-pack` là chính xác để cùng một commit cho ra cùng các byte trên mọi máy. Chỉ có tên được phát ra; giá trị biến môi trường, giá trị hằng và giá trị thành viên enum không bao giờ được phát.

Hai thuộc tính bảo mật. (1) Extractor integrations phát ra **khóa, không bao giờ giá trị**: file env duy nhất nó mở là `.env.example`, `.sample` hoặc `.template`, không bao giờ là `.env` thật, và pattern của nó không có nhóm bắt quanh giá trị, nên không có gì để rò rỉ. Khối `environment:` của compose được đọc, nhưng chỉ khóa được giữ lại. (2) Đầu vào schema SQLite **chỉ qua `--db`, không bao giờ tự suy ra**: một file `.db` hay `.sqlite` lạc trong repo không bao giờ được đọc trừ khi bạn nêu tên, nên nó không thể tình cờ trở thành tri thức công ty được publish.

`struct.coverage` cho SA biết nên tin vào sự vắng mặt tới đâu: một bảng, `Area | Scope | Framework/language | Status | Detail`, cho `routes`, `pages`, `jobs`, `symbols`, `files` và `schema`.

Trạng thái	Ý nghĩa
`complete`	Mọi dạng nhận ra được trong phạm vi đó đều đã phân giải
`partial`	Có thứ được thấy nhưng chưa phân giải (đường dẫn route động, file không parse được); mỗi vị trí được nêu dưới dạng `file:line`
`not scanned`	Chưa nhìn vào gì; phần detail nói lý do (thiếu extra, chưa có adapter)

`jobs` không bao giờ là `complete`: một job có thể nằm ở nơi không adapter nào nhìn tới (`pg_cron`, `setInterval` viết tay, crontab), nên dòng của nó luôn là `partial` với detail `jobs outside framework schedulers are not detectable`. Với một route, page, symbol, file hay partition, index hoặc primary key của bảng, SA chỉ viết một thứ mới là mới khi phạm vi đó `complete`: `<thing> in <path> [NEW: D<n>]` cho route, page hoặc symbol, và `<path> [NEW: D<n>]` làm một mục `Files:` riêng cho file. Nếu không, đó là một mục `Open decisions` trích dẫn trạng thái, và `kb ticket check` thực thi điều này.

Giới hạn: `schema` không hiện thực dialect SQL nào; câu lệnh nó không áp dụng được làm phạm vi thành `partial` và nêu `file:line`. Nếu `.kb/<doc_id>/` chứa một tài liệu mà `kb code-ingest` không viết ra, nó thoát với mã 1 và không ghi gì (hãy chuyển tài liệu đi hoặc chọn `--doc-id` khác). Cây bẩn in cảnh báo `dirty_tree`.

6.4 Cái gì được thực thi, cái gì chỉ được yêu cầu

Quy tắc	Thực thi bởi
PR mang bằng chứng của nó	`kb pr lint` trong `kb-pr-lint.yml` ở mọi PR: `## Ticket`, `## Changes`, `## Testing` được điền, lệnh test đã dán, một dòng `Blocking: No`
Tri thức chưa review không tới được hub	`kb build` thoát 1 khi còn section `-svc` nào `pending`
Cache ngữ cảnh do CLI sở hữu	`kb resolve --status-only --cache` từ chối cache có version, tham chiếu hay khối khác

`kb pr lint` nhận một file hoặc stdin và không bao giờ nhận một chuỗi làm tùy chọn: mô tả PR là văn bản do kẻ tấn công kiểm soát được, và giữ nó ngoài argv ngăn người gọi nhúng nó vào một lệnh shell.

Mọi thứ khác (test-first, bằng chứng xác minh, GATE 1 và GATE 2, A1 đến A4) là kỷ luật qua prompt mà không gì trong `kb` đo được; người kiểm tra là bạn, tại cổng. Chỉ A5 có răng, qua dòng `Blocking:`. Test-first có một ngoại lệ có giới hạn: các lớp thay đổi không có hành vi quan sát được, được nêu trong `docs/tdd-exemptions.md` và khai báo trong plan khi plan được viết.

6.5 Quyền của chế độ auto

`kb init --kind dev` gộp các quyền của nó vào `.claude/settings.json` của bạn. **Cho phép:** `git` thông thường (add, commit, status, diff, log, fetch, pull, switch sang `auto/*`, `fix/*`, `hotfix/*`, branch, worktree, merge, `push -u origin HEAD`), `gh pr` và `gh run`, `Read`/`Edit`/`Write`, các công cụ MCP `strata-kb`, nhiều lệnh con `kb`, và các lệnh test, install, lint, build của `langs` bạn. **Từ chối:** mọi `git push` nêu `main` hay `master`, force-push, refspec `+` và `:` cùng `--delete`; các dạng `git checkout`, `switch` và `branch` chuyển tới, tạo, đổi tên hay xóa `main` hoặc `master`; `git reset --hard`; `gh repo delete`, `gh run delete`; `kb init`; `rm -rf /*`; việc sửa `.claude/settings.json`, `.claude/settings.local.json` và `.git/**`. Các hook `Stop` và `SubagentStop` chạy `kb usage ingest-transcript --hook-stdin` cho báo cáo chi phí.

Báo cáo chi phí. Hook thêm khoảng 0,6 giây mỗi lượt (chi phí khởi động `kb`). `kb usage report` ghi `.kb/usage/report.html`, theo ticket, phase và model; với `--ticket <id> --md` nó in một bảng Markdown. Hook lỗi thì im lặng theo thiết kế (hook `Stop` không thể chặn một lượt kết thúc), nên nếu `.kb/usage/*.jsonl` không tăng, hãy đọc `.kb/usage/ingest-errors.log`: đó là nơi duy nhất lỗi bên trong `kb` hiện ra. Ingest lại một transcript đã kết thúc là an toàn, vì lệnh gọi đã có trong sổ cái thì bị bỏ qua: `kb usage ingest-transcript <path>` nạp bù một transcript. Chỉ chạy nó trên phiên đã kết thúc: một lệnh gọi còn đang stream sẽ bị lưu với số token đầu ra chưa đầy đủ, và một dòng đã lưu không bao giờ thay đổi. `report.html` được tạo lại từ sổ cái ở mỗi lần chạy, nên sổ cái mới là bản ghi, không phải báo cáo. **Phân tầng model** chỉ là khuyến nghị, không có gì trong `kb` bắt buộc. Giá model lấy từ `usage-prices.yaml` (đi kèm gói, ghi đè theo từng model ở `.kb/usage-prices.yaml`), có `effective_date`. Model không có dòng giá được báo là `unpriced`, không bao giờ đoán, và các lệnh gọi fast-mode được tính theo giá chuẩn và đánh dấu, nên tổng có chứa chúng chỉ là cận dưới. Đổi model ở ranh giới phase, không bao giờ giữa một phase: đổi model làm mất prompt cache và phải trả một lần ghi cache mới, nên vài lượt model rẻ giữa một phiên model mạnh có thể tốn **nhiều hơn** không phân tầng. Kiểm tra kết quả bằng `kb usage report`, vốn chia chi tiêu theo model.

6.6 Quy tắc Q&A của hub liên quan tới bạn

Câu hỏi và câu trả lời nằm trong kho Q&A của hub, chỉ được ghi qua `kb_question_ask` (Dev) và `kb_question_answer` (BA). `asked_by` và `answered_by` là tên do người gọi tự gõ (các skill truyền `git config user.name`): để theo dõi, không phải xác thực. Giới hạn: 4000 ký tự cho câu hỏi hoặc câu trả lời, 1000 cho context, 100 cho tên, tối đa 50 câu hỏi mở cho mỗi ticket. Các công cụ MCP mà skill của bạn gọi: `kb_search`, `kb_get_section`, `kb_context_new`, `kb_resolve`, `kb_ticket_lint`, `kb_ticket_check`, `kb_ticket_fetch`, `kb_question_ask`, `kb_question_list`.

6.7 Tên dành riêng

`<repo_id>-code` và `<repo_id>-svc` được dành riêng cho tri thức về code của chính repository này; tài liệu nghiệp vụ không bao giờ được dùng hai hậu tố đó. Tiền tố section là một hợp đồng đã công bố:

Tiền tố	Tài liệu	Chủ sở hữu
`struct.tree`, `struct.files.<dir>`, `struct.coverage`, `db.<table>`, `dep.<eco>`, `int.<name>`, `api.<tag>`, `route.<slug>`, `page.<slug>`, `job.<slug>`, `sym.<slug>`, `cmd.<purpose>`	`-code`	extractor
`svc.<name>`	`-code` và `-svc`	extractor (bằng chứng) và bạn (trách nhiệm do model soạn)
`flow.<name>`	`-svc`	bạn
`hist.<name>`	`-svc`	chỉ `kb svc note`